



BUIDL CTC 2026 · ATTESTCOIN PROTOCOL

Deadswitch

The position dies when the collateral leaves.

Collateral on Ethereum, debt on Creditcoin, proven by an Attestcoin receipt from the source chain.

No bridge. No relayer. No price feed.

Cross-chain lending has to answer one question.

Is the collateral still there?

Every design today answers it with a trusted third party: a bridge, an oracle committee, or a relayer. That party becomes both the thing you have to trust and the thing that gets attacked.

- A bridge moves value. It cannot tell you whether collateral moved.
- An oracle committee can report it, but then the loan rests on the committee.
- A price feed prices the asset. It does not prove the asset is present.

The vault never blocks a withdrawal. It logs it.

ETHEREUM SEPOLIA

`CollateralVault.withdraw()` emits
`CollateralWithdrawn(positionId, amount, remaining)`.

The vault cannot know the debt. It guarantees only that every withdrawal is provable.

CREDITCOIN CC3

Anyone submits an inclusion and continuity proof to
`DeadswitchManagerV3.execute()`.

The `0x0FD2` precompile verifies it synchronously, in that same transaction. Below threshold, the position liquidates.

Proof submission is permissionless. Trust comes from the attester quorum, not from whoever submitted it.

Position #3, killed by the keeper.

COLLATERAL, SEPOLIA	
40 TST	
Before	100 TST
Withdrawn	60 TST

POSITION, CREDITCOIN	
Liquidated	
Threshold	50 TST
Attested	40 TST

Verified on Blockscout. 0xd149012c...78d6 decodes to CollateralAttested(3, 40.0) and PositionLiquidated(3, 40.0, 50.0). No human in the loop.

The replay key omits action.

```
queryId = keccak256(chainKey, blockHeight, txIndex)
// action is not in it
processedQueries[queryId] = true; // burned before dispatch
```

One transaction carrying a deposit and a withdrawal, submitted as a deposit, makes the withdrawal permanently unprovable.

```
v2 submitted as action=1 -> attested 100.000000000000000001, ACTIVE
v2 same tx as action=0 -> reverts "Query already processed"
the collateral is gone; the position reports healthy, forever
```

The guard becomes the weapon.

Consumers read `logs[0]` and revert on the emitter check. Prefix a decoy event from a throwaway contract in the same transaction, and the genuine withdrawal can never be processed.

THE GUARD HOLDS

A forged event from a fake vault bounces off with "Event not emitted by registered vault".

THE SAME GUARD KILLS

A decoy log at index 0 makes that same string reject the real withdrawal, permanently.

A guard that rejects a whole transaction because of one log it does not own is a denial-of-service primitive.

DeadswitchBase.sol

- **Action derived from each log's topics[0]**, never trusted from the caller. A submitter cannot choose which half of a transaction is seen.
- **Every log from the registered vault applied, in log order.** One transaction with a deposit and a withdrawal produces both transitions, correctly sequenced.
- **Foreign logs skipped, never reverted on.** A decoy cannot censor a genuine event.
- **blockHeight threaded into the handler** and stored per position, so a stale proof cannot overwrite newer attested state.

Both attacks were re-run against v3 on live chains. Both positions liquidated correctly.

EVIDENCE

Everything is on public testnets.

CONTRACT	CHAIN	ADDRESS
DeadswitchManagerV3	Creditcoin CC3	0x44e2d55Af74f400b97fBC010Acd504A1458bA682
CollateralVault	Sepolia	0x80366d27b907828A36243140ce6ACED6350EE412
NaiveManager (control)	Creditcoin CC3	0x9EdeA943Bc77caF9cB892077575E2d7E7f2B5142

- Current CC3 contracts are source-verified on Blockscout. Every event decodes publicly.
- Both findings filed upstream: [gluwa/USC-Build-Examples#37](#)
- Clone and run: `yarn install && yarn server`. Position #4 is staged live.



SCOPE, STATED PLAINLY

One collateral asset. One source chain. Liquidation and restore.

Positions are owner-registered. There is no lender market, no interest accrual, no auctions. What we are submitting is the trust-minimised liquidation primitive, and the security work around it.

nuel-osas.github.io/deadswitch